

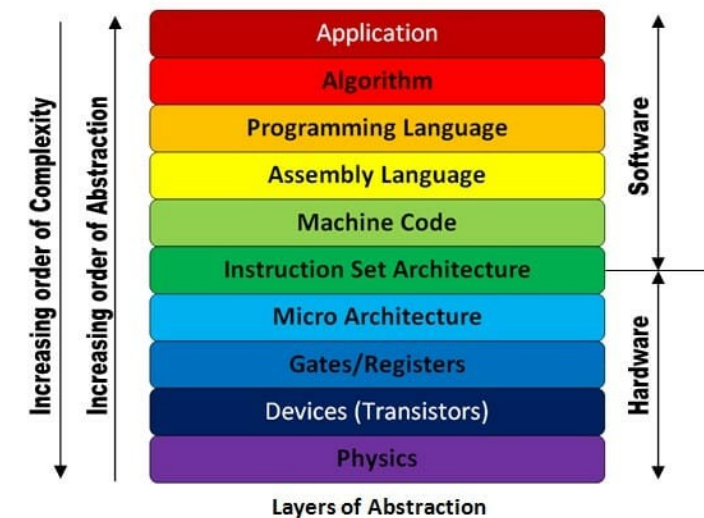


CIS-310: Computer Organization & Assembly Language

Introduction

MW 10:00-11:45AM

Dr. Probir Roy

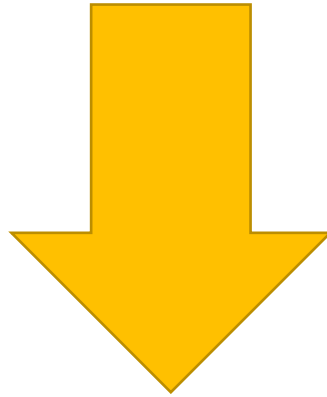


High-level programming language

- C/C++
- Java
- GoLang
- Rust
- C#
- Python

1. How these high-level languages run on computer?
2. How computers understand these languages?

High-level language



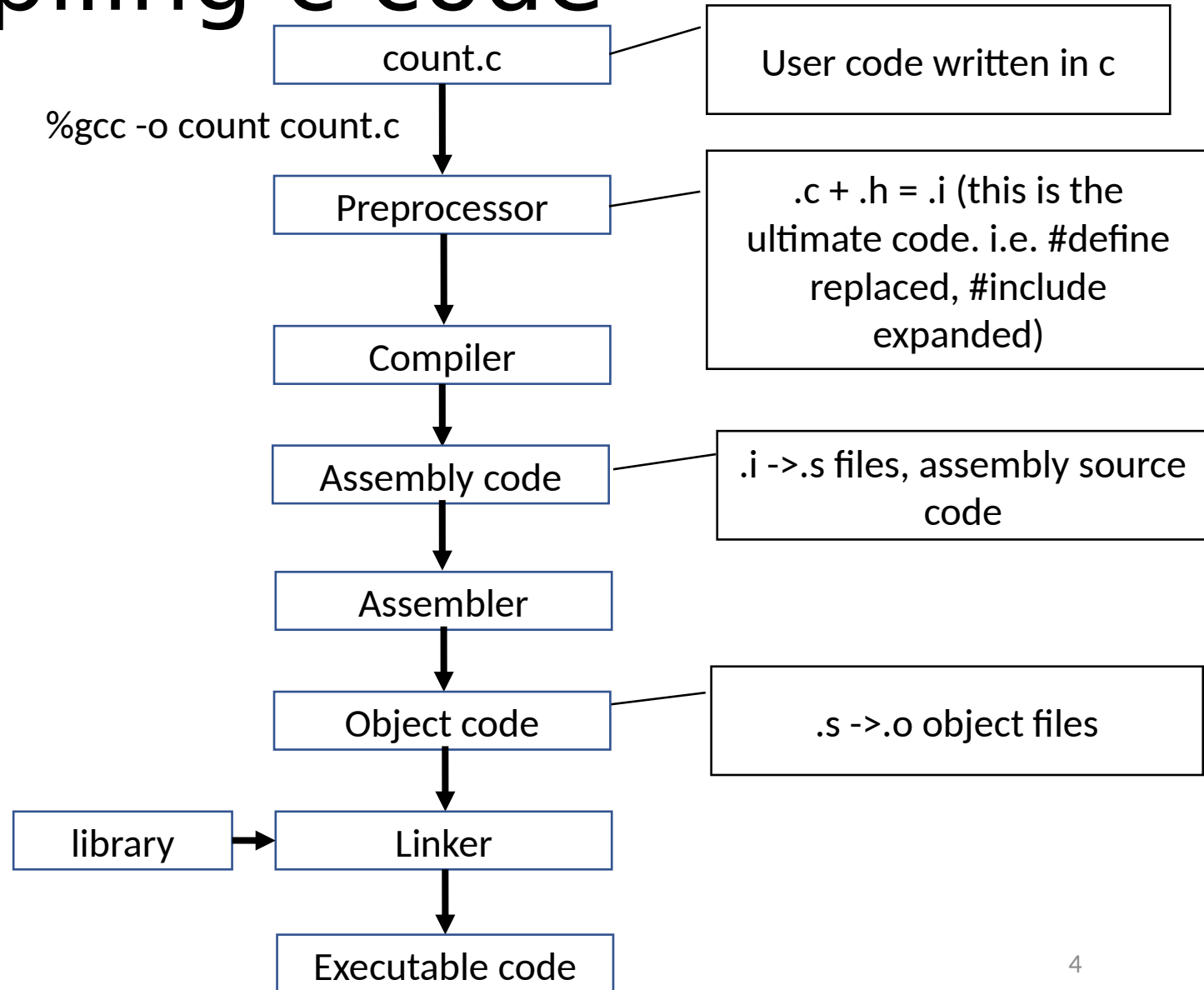
Translation

Hardware (CPU, Memory etc.)

Example: Compiling c code

```
// Count application
#include <stdio.h>
void main(void)
{
    int count;
    // Display the numbers 1 through 9
    for(count = 1; count <= 9; count++)
        printf("%d ", count);
    printf("\n");
}
```

%gcc -o count count.c



Software

%gcc -o count count.c

count.c

Preprocessor

Compiler

Assembly code

Assembler

Object code

library

Linker

Executable code

Hardware

Instruction set architecture

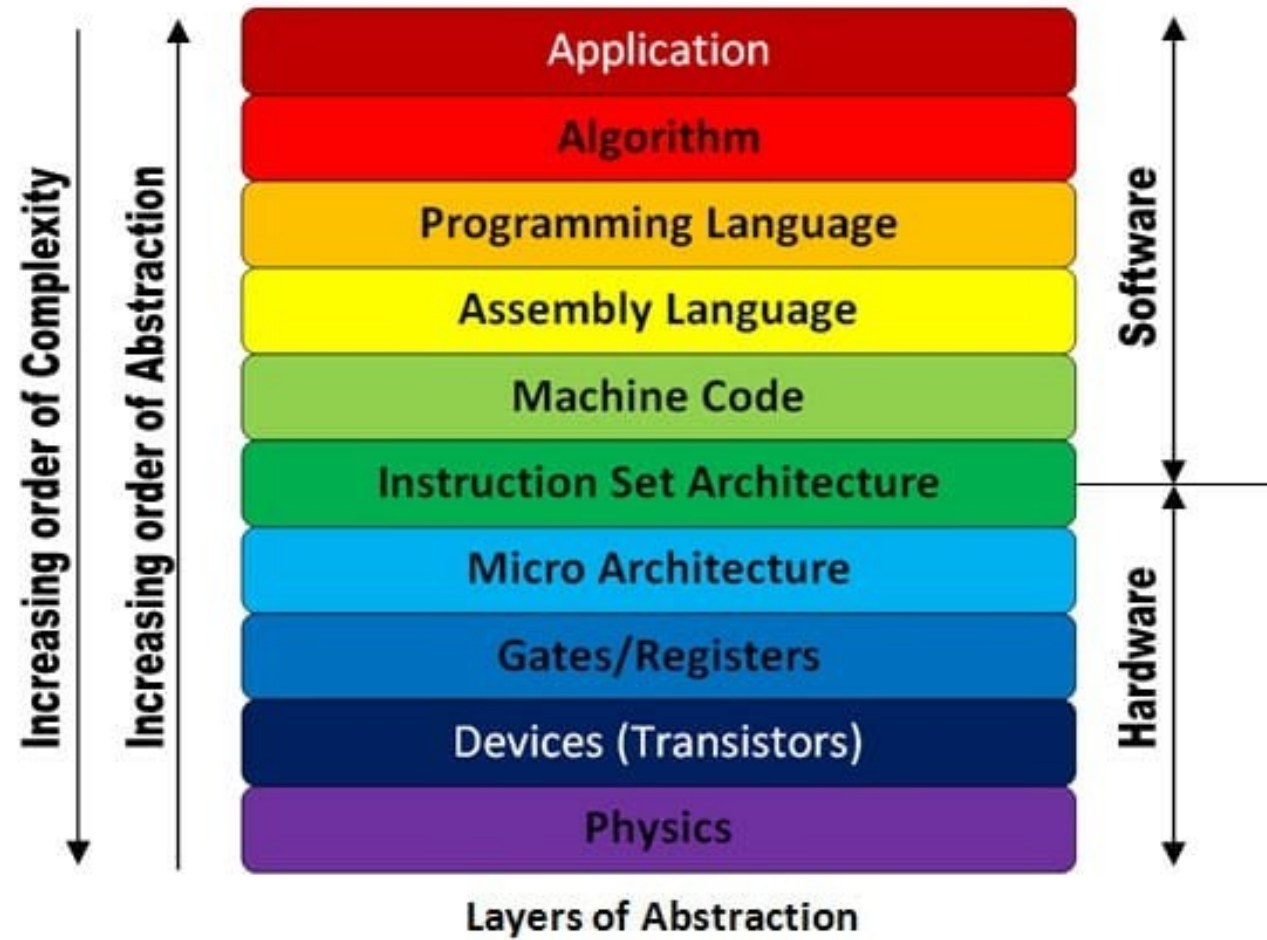
Micro-architecture

Logic-gates

CIS 474 Compiler Design

CIS 310 Computer organization and assembly language

Question: Why do we need so many layers to execute a source code?

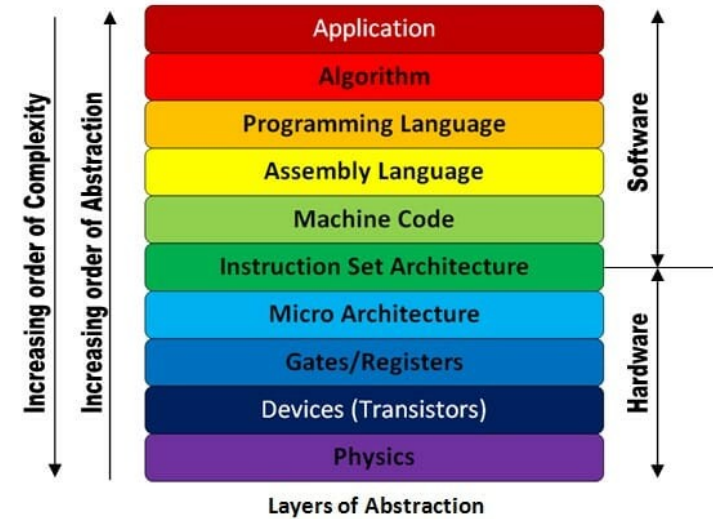


Assembly language

- What is it?
 - A low-level programming language
 - Example: program prints the number 1 to 9 on the screen

- <https://docs.google.com/document/d/1A1rKnVwboGxivs7dHAIv4QMUXklKnKoIUA2dIW2PCX0/edit?usp=sharing>

- Processor dependent
- Writing in assembly language is challenging

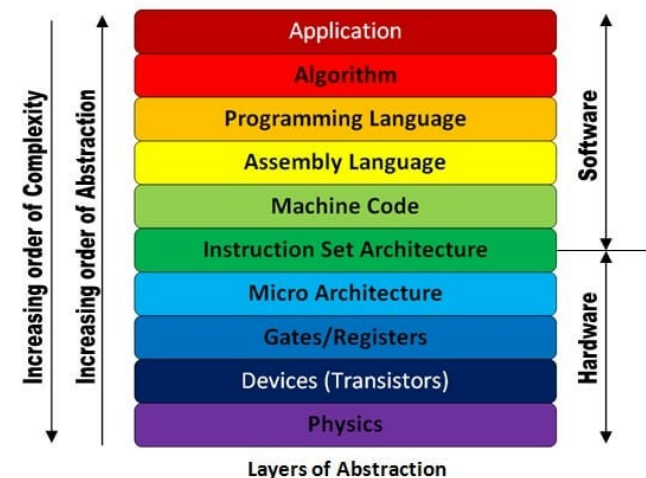


Why we should learn assembly language?

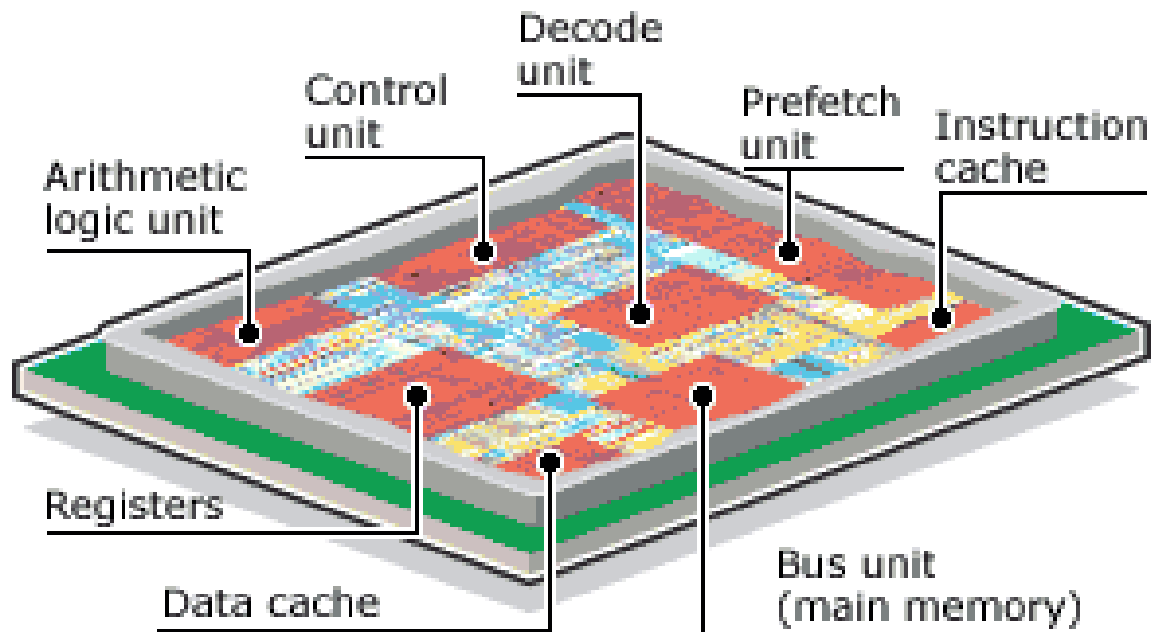
- Compilers can be buggy- we may need to verify from generated assembly code.
- Assembly languages allow fine-grain manipulation of hardware
 - Device driver or exception/interrupt handler (OS engineers)
- Allows to write optimized code which might be difficult to write in high-level language (Linear algebra with BLAS)
- Often manually written assembly code require less space than that generated from high-level language
 - Why important?
 - Embedded systems might have space constraint
- Working on new architecture (may not have compiler support for high-level language)
 - GPU, Tensor processing unit (TPU)
 - New hardware features (Single Instruction multiple data (SIMD))
- Real-time system, cryptography, reverse-engineering, etc.

Processor Architecture

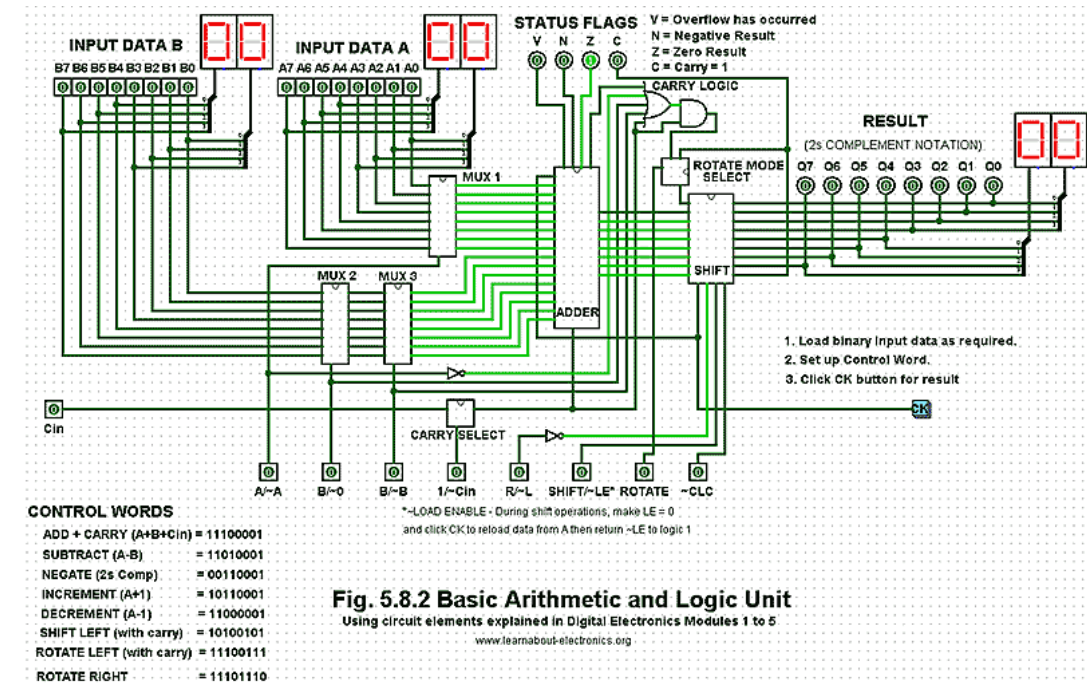
- Instruction set architecture (ISA)
 - An abstract model of processor architecture
 - Defines the basic operations (add, multiply, move)
 - Chip manufacturers realize this abstract models to create a processor.
 - Example: x86 ISA
 - Popular processors that are based on x86 ISA:
 - Intel 8086, Intel Core 2, Intel Skylake, Kaby Lake and Cannon Lake series, AMD Ryzen etc.
 - Since all these processors are based on x86 ISA, binary compatibility is easily achieved between generations of processor.
 - What are the other types?
 - RISC family: Power ISA, MIPS
 - Popular processor: Power9 (Summit), ARM processors (smartphones)
 - An x86 machine Instruction / opcode, Example
 - B8 00000004
 - Equivalent assembly: mov eax,4
 - <http://ref.x86asm.net/coder64-abc.html>



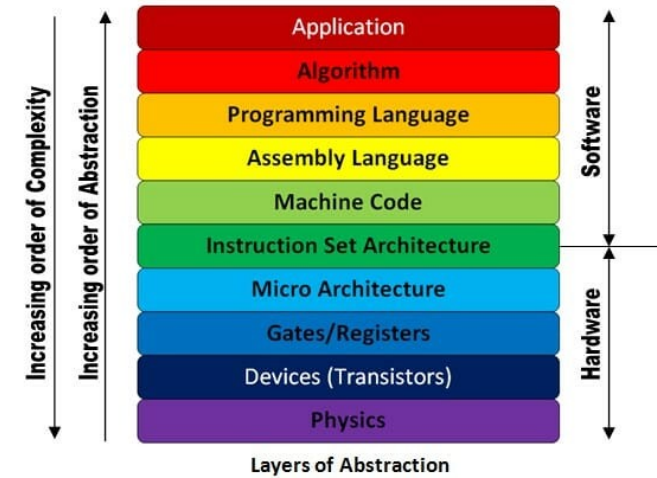
Micro-architecture



Arithmetic logic unit (ALU)



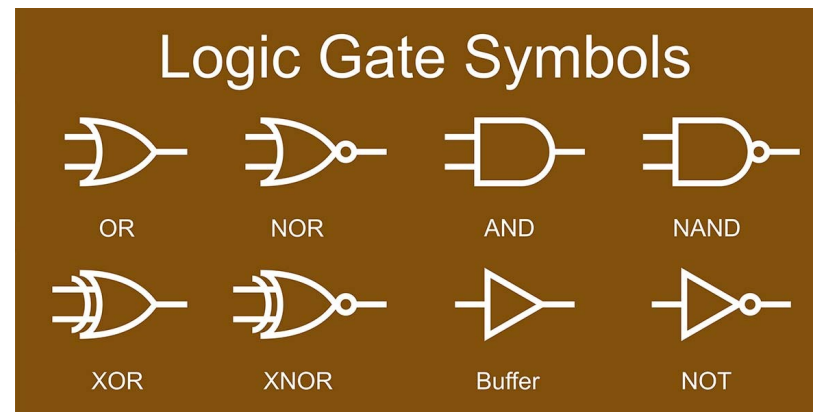
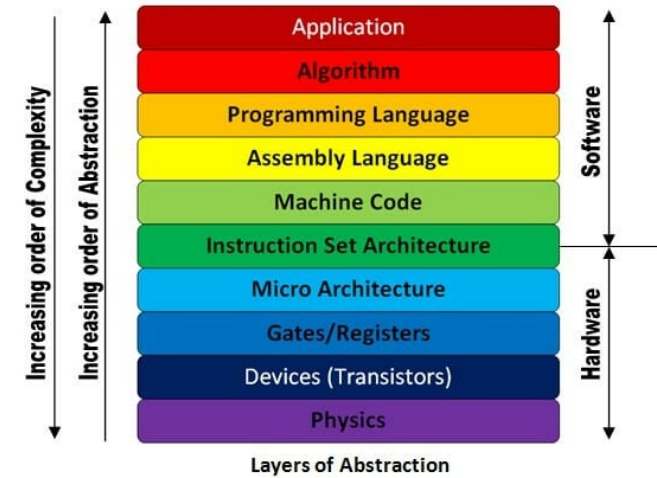
Logic circuits



- Arithmetic logic unit
 - that performs arithmetic and bitwise operations on integer **binary numbers**
 - Adder , shift circuit

Digital logic design

- Logic gates
 - the basic building blocks of any digital system
 - implements a Boolean function
 - it performs a logical operation on one or more binary inputs and produces a single binary output



Expectations – (Assembly language)

- Assembly is complicated- you don't need to memorize the manual.
- However, you need to be able to
 - Understand and explain the basic constructs of assembly
 - Demonstrate proficiency to read code written in assembly language and explain the implementation logic
 - Needs to be able implement simple application logic using assembly

Expectations – (Computer organization)

- Understand the basic components of a computer (CPU, RAM, Cache, Processor pipelining, BUS, clock)
- Understand and explain how processor executes a code
- Understand and explain logic gates
- Understand and implement basic circuit functionalities using logic gates

Syllab us

Data representation

Data representation

- Binary integer
 - Unsigned binary integer
 - Binary to decimal conversion
 - Decimal to binary conversion
 - Binary Addition
- Integer storage size
- Hexadecimal integer
 - Why we use hexadecimal integer
 - Hex to decimal conversion
 - Decimal to hex conversion
 - Hex addition
- Signed binary integer
 - Two's complement
 - Why we use two's complement
 - Hex two's complement
 - Signed binary to decimal
 - Signed decimal to binary
 - Signed decimal to hex
 - Signed hex to decimal
 - Max & min values
- Binary subtraction
- ASCII

Unsigned binary number

- a number expressed in the base-2 numeral system
- In binary number, numbers are represented with 0 and 1
 - Example: 1011 1000
- Comparatively, a decimal number expressed in base-10 numeral (0 to 9)
- MSB (Most significant bit)
- LSB (Least significant bit)

MSB -> [1]011 100[0] <- LSB

Converting a unsigned binary to decimal

- Example 1:
 - Binary representation: 1000
 - Decimal representation: 8
- Example 2:
 - Binary representation: 1001 1000
 - Decimal representation: 152
- How to convert binary <-> decimal

Unsigned binary to decimal

- 1) Consider bit position.
- 2) Calculate bit weight for a given bit position n (equation: 2^n)
- 3) Multiply bit weight with bit value
- 4) Add the results

Binary	1	0	0	1	0	1	0	1	1	0	1	
Decimal	$1 \times 2^{10} +$	$0 \times 2^9 +$	$0 \times 2^8 +$	$1 \times 2^7 +$	$0 \times 2^6 +$	$1 \times 2^5 +$	$0 \times 2^4 +$	$1 \times 2^3 +$	$1 \times 2^2 +$	$0 \times 2^1 +$	$1 \times 2^0 =$	1197

Unsigned decimal to binary

- Divide the decimal integer and the quotient by 2. The remainder of each operation determines the binary value on that bit position
- Example: convert decimal 16 to binary 10000

_____ remainder = 1

2) 1

_____ remainder = 0

2) 2

_____ remainder = 0

2) 4

_____ remainder = 0

2) 8

_____ remainder = 0

2)16

<https://www.khanacademy.org/math/algebra-home/alg-intro-to-algebra/algebra-alternate-number-bases/v/decimal-to-binary>

Unsigned binary addition

- Start from LSB, add bit values, add **carry** if available:

$$\begin{array}{r} 01011110 \\ + 01110111 \\ \hline 11010101 \end{array}$$

Unsigned binary addition

$$1\ 1\ 1\ 0 + 1\ 1\ 1\ 1 = ?$$

Hexadecimal number

- Why we use hexadecimal integer?
 - Binary numbers are cumbersome. Requires large number of bits to represent.
 - Since, hexadecimal numbers are power of 2, they are easier to represent compared with decimal numbers

Hexadecimal number

Binary	Decimal	Hexadecimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F

Hexadecimal number to binary and decimal

- Hexadecimal number:
A816

Convert to binary:

Convert to decimal:

Hexadecimal number to binary and decimal

- Hexadecimal number:
A816

Convert to binary:

$$0xA816 = 1010\ 1000\ 0001\ 0110$$

Convert to decimal:

$$\begin{aligned} 0xA816 &= A \times 16^3 + 8 \times 16^2 + 1 \times 16^1 + 6 \times 16^0 \\ &= 10 \times 16^3 + 8 \times 16^2 + 1 \times 16^1 + 6 \times 16^0 \\ &= 10 \times 4,096 + 8 \times 256 + 16 + 6 \\ &= 43,030 \end{aligned}$$

Convert decimal to hexadecimal

- Decimal number : 422 = 1A6 hexadecimal
 - How to do it?

Convert decimal to hexadecimal

- Decimal number : 422 = **1A6** hexadecimal

Division	Quotient	Remainder
422/16	26	6
26/16	1	A
1/16	0	1

Hexadecimal addition

- Example:

$$\begin{array}{r} A A \\ + A A B \\ \hline C 5 5 \end{array}$$

Hexadecimal Subtraction

- Example:

$$\begin{array}{r} \text{AB1} \\ - \text{1AB} \\ \hline \text{906} \end{array}$$

Topics covered today

- Overview of computer organization and assembly
 - Abstraction
- Data representation
 - Binary number
 - Hexadecimal number
 - Conversion between binary, hexadecimal and decimal
 - Arithmetic operations
 - Binary Addition
 - Hexadecimal addition